Amber is also currently available as an arch user repository (AUR) package in Arch Linux and NixOS. As time progresses, it should be available in other distributions too.

Usage

The '--help' flag is pretty self-explanatory, as can be seen in the code below:

```
> amber --help
amber 0.1.1

Utility to store encrypted secrets in version trackable plain
text files

USAGE:
    amber [FLAGS] [OPTIONS] <SUBCOMMAND>

FLAGS:
•  -h, --help        Prints help information
•  --unmasked    Disable masking of secret values during exec
•  -v, --verbose     Turn on verbose output
•  -V, --version     Prints version information

OPTIONS:
    --amber-yaml <amber-yaml>    amber.yaml file location [env:
AMBER_YAML=] [default:
              amber.yaml]
SUBCOMMANDS:
    encrypt: Add or update a secret
    exec: Run a command with all of the secrets set as
environment variables
    generate: Generate a new strong secret value, and add it to
the repository
    help: Prints this message or the help of the given
subcommand(s)
    init: Initialize a new directory
    print: Print all of the secrets
    remove: Remove a secret
```

Next, let's create a Git repository to show a demo of how to use Amber:

```
> mkdir amber-demo
> cd amber-demo/
> git init

Initialized empty Git repository in /home/sibi/github/amber-
demo/.git/
```

Let us now initialise 'amber' to get a secret key:

```
> amber init
Your secret key is: 4a3f13b65fff6d575cbd4acba30861b5b5e
f992a24548f314936453cbad9dccc
Please save this key immediately! If you lose it, you will lose
access to your secrets.
Recommendation: keep it in a password manager
If you're using this for CI, please update your CI configuration
with a secret environment variable
export AMBER_SECRET=4a3f13b65fff6d575cbd4acba30861b5b5ef992
a24548f314936453cbad9dccc
```

It is better to save this key in a password manager. If you are planning to integrate it in the CI systems like GitHub, the above variable has to be added using their Web interface. Since this demo is being shown in the command line interface (CLI), we will just export it in our current shell session:

```
> export AMBER_SECRET=4a3f13b65fff6d575cbd4acba30861b5b5ef992a24548f31
4936453cbad9dccc
```

The environment variable 'AMBER_YAML' can be used to specify the custom location 'amber.yaml' where the encrypted secret file will be stored. Its default value is 'amber.yaml'.

Let us now create a secret value. The name of the secret variable will be 'SECRET_VAR1', and it will hold 'hello' as its secret value.

```
> amber encrypt SECRET_VAR1 hello
```

This is all that was needed. If the file 'amber.yaml' is looked at now, a change in the content can be noticed:

```
yaml
> cat amber.yaml
---
file_format_version: 1
public_key: 5a9f77901faa095dd500bbe1dc2ed25e20ba7c1310f835
fd6be2b6692877441d
secrets:
  - name: SECRET_VAR1
    sha256: 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e
5c1fa7425e73043362938b9824
    cipher:
dd5135acae1c180b416514726208e113a7dd0976ec75b4ca8bc5
69654af7d9367e5e3c36eec06ffeedd736413ae0c1fca02f8cb7ad
```

The secret value can also be printed using the following code:

```
> amber print
export SECRET_VAR1="hello"
```

Continued on page...75